TGT: Temporal Gait Transformer for Analyzing Biomechanics of Walking Abnormalities

Sebastian Kot*
City, University of London

Mohammed Kairmi*
City, University of London

*Equal contribution

Abstract

We present the Temporal Gait Transformer (TGT), a classification model that takes raw smart insole sensor data and identifies five distinct walking activities: normal walking, injury walking, stepping, swaying, and jumping. The input data comes from pressure sensors, accelerometers, and orientation quaternions on both feet. After merging left and right insole streams into synchronised rows at 60 Hz, we segment the recording into overlapping windows of 80 timesteps (1.33 seconds, roughly one full gait cycle) with a stride of 30. Each timestep carries 50 sensor features. Every window is independently z-score normalised per feature to remove subject-specific sensor baselines. The TGT uses a two-layer 1D convolutional stem (32 channels) to extract local temporal patterns such as heel-strike and toe-off phases, then passes the result through two Transformer encoder layers with two-head self-attention to capture longer-range gait dynamics. The output is obtained by mean-pooling across timesteps rather than a learnable classification token, which we found more stable given limited training subjects. The model is trained with label-smoothed weighted cross-entropy loss, mixup augmentation, and the AdamW optimiser with cosine annealing. Aggressive data augmentation (Gaussian jitter, per-channel scaling, channel dropout, temporal masking, and random time shifts) is applied during training to simulate cross-subject variability and reduce overfitting. All data splits are performed at the file level so that overlapping windows from the same recording never appear in both training and validation sets.

1 Data Preprocessing

1.1 Data Types

Data is collected from smart insoles on both feet of each subject. At every time step, the insoles measure pressure at multiple locations and provide accelerometer, gyroscope, and magnetometer readings, along with a computed quaternion and an insole label (right or left). Redundant fields include internal time stamps for each sub-device and battery status. A detailed list of all per-timestep features is provided in Table 4 in the Appendix.

1.2 Separating Input Data

The raw data for each walking condition is first separated into right and left insole streams using the insole identifier *ele_36*. We then compare the time stamps of rows from the right and left insoles and match them into a single combined row when the difference in recording times satisfies the constraint $\delta t < 0.02\,\mathrm{s}$. Rows that do not satisfy this constraint are discarded. We remove redundant sub-time fields and drop gyroscope and magnetometer readings, because this information is already captured by the quaternions. The resulting dataset, with all retained channels, is summarised in Table 5 in the Appendix. Each remaining row is a full entry for both feet, containing 50 features in total: 18 pressure values, 3 accelerometer axes, and 4 quaternion components per foot.

1.3 Sampling Rate

After merging, the data is resampled to a fixed 60 Hz. If the merged stream has a frame rate below 60 Hz we upsample by linear interpolation. If it is above 60 Hz we downsample by

selecting evenly spaced indices. This ensures that every recording, regardless of the original device clock, produces a uniform 60 samples per second stream.

Note on upsampling correction. An earlier version of the upsampling routine generated evenly spaced target timestamps and called `reindex` directly on the original data frame. Because the target timestamps almost never coincided exactly with the recorded timestamps (floating-point mismatch on ten-digit Unix epochs), the operation discarded virtually all original samples and left a single anchor point, causing `interpolate` to replicate one constant row across the entire recording. This affected 35 of 92 Injury_walking training files and all four Group 10 recordings, whose native frame rate fell below 60 Hz. The corrected implementation first takes the union of the original and target timestamp indices, interpolates over the combined index so that every original measurement serves as an anchor, and then selects the target timestamps. After this fix, all upsampled recordings contain the expected temporal variation.

1.4 Windowing

We define a window parameter W as the number of consecutive rows provided to the model. In our configuration we use $W = 80$, which corresponds to 1.33 seconds at 60 Hz, enough to cover a full gait cycle with some margin. The stride between consecutive windows is set to $S = 30$, giving 62.5% overlap. This overlap produces more training samples and ensures that transient gait events are captured even if they fall near a window boundary. Each window is therefore a matrix of shape 80×50.

1.5 Normalisation

We apply two layers of normalisation. First, a global z-score is computed per feature across the entire training set and applied to all data (training, validation, and evaluation). The mean and standard deviation are fit only on training windows to prevent information leakage. This brings all features onto a comparable scale.

Second, each window is independently z-score normalised per feature. For each window $\mathbf{X} \in \mathbb{R}^{W \times F}$, we compute the mean μ_f and standard deviation σ_f of each feature f across the W timesteps and normalise:

$$\hat{x}_{t,f} = \frac{x_{t,f} - \mu_f}{\sigma_f + \varepsilon} \tag{1}$$

where $\varepsilon = 10^{-8}$. This removes subject-specific absolute sensor baselines (e.g. resting pressure levels and accelerometer offsets that vary with body weight, foot placement, and device calibration) so that the model learns from the temporal *shape* of each window rather than its absolute magnitude.

1.6 Attaching Outcomes

For each input window we attach a walking label from the following set of classes, summarised in Table 1:

1.7 Split

Because we use overlapping sliding windows, a naive random split would place near-identical windows into both the training and validation sets, inflating validation scores. To avoid this data leakage, we split at the *file* level: all windows from the same CSV recording are assigned to the same partition. We use a 90/10 train/validation split. The validation set is used during training to monitor generalisation and select the best checkpoint via early stopping. All available data, including the Group 10 recordings, is included in the training pool to maximise subject diversity. Final evaluation is performed on a completely separate set of recordings not seen during training.

2 Methodology

The TGT is a lightweight hybrid CNN-Transformer model. A 1D convolutional stem first extracts local temporal features from the raw sensor window into 32-dimensional representations, then a Transformer encoder captures longer-range dependencies across the gait cycle. The output is produced by mean-pooling across all timesteps rather than a learnable classification token, which we found more reliable when training data comes from only a handful of subjects. We describe each stage below.

2.1 1D Convolutional Stem

The input to the model is a window of shape $W \times 50$, where each of the $W = 80$ rows is a single timestep with 50 sensor features. Rather than projecting each timestep independently through a linear layer, we first transpose the input to shape $50 \times W$ and apply two 1D convolutional layers along the time axis:

The first convolution uses a kernel size of 7 and maps the 50 input channels to $d_{\text{model}} = 32$ output channels. The second convolution uses a kernel size of 5 and maps 32 channels to 32. Both use same-padding so the temporal length stays at $W = 80$. Each convolution is followed by batch normalisation, a GELU activation, and dropout.

The motivation for a convolutional stem is that gait signals contain local temporal patterns (the pressure wave from heel-strike to toe-off, the acceleration peak at foot contact) that are best captured by filters operating over small neighbourhoods. These local features tend to be more consistent across subjects than the raw sensor magnitudes, which vary with body weight, sensor placement, and foot shape. After the stem, we transpose back to shape $W \times 32$ for input to the Transformer.

2.2 Positional Encoding

Since the Transformer has no built-in notion of ordering, we add a fixed positional encoding to each token so the model can distinguish early timesteps from later ones within the window. For a given position $\text{pos} \in \{0, 1, \ldots, W-1\}$ and a dimension index $i \in \{0, 1, \ldots, 15\}$ we compute:

$$PE_{\text{pos},\,2i} = \sin\left(\frac{\text{pos}}{10000^{\,2i/d_{\text{model}}}}\right) \tag{2}$$

$$PE_{\text{pos},\,2i+1} = \cos\left(\frac{\text{pos}}{10000^{\,2i/d_{\text{model}}}}\right) \tag{3}$$

with $d_{\text{model}} = 32$. Lower dimensions oscillate rapidly and capture fine-grained local timing, while higher dimensions oscillate slowly and encode where the timestep sits within the segment. The result is a 80×32 matrix of positional vectors added element-wise to the convolutional features.

2.3 Transformer Encoder

The core of the model is a stack of $N = 2$ identical Transformer encoder layers. Each layer consists of two sub-blocks.

2.3.1 Multi-Head Self-Attention

For each of the 80 tokens, the layer computes Query (Q), Key (K), and Value (V) vectors via learned linear projections. The attention score between any two tokens is the scaled dot product of one token's query with another's key. These scores form an 80×80 matrix, passed through a softmax so that each row sums to one. Multiplying by the Value matrix produces a new representation where each token is a weighted combination of all tokens, according to their relevance to it.

We use $H = 2$ attention heads in parallel. Each head operates on a $32/2 = 16$ dimensional slice of the embedding, and the results are concatenated back to 32 dimensions.

2.3.2 Feed-Forward Network

After attention, each token passes independently through a two-layer feed-forward network (FFN). The FFN projects from 32 to 128 dimensions (a $4\times$ expansion), applies a GELU activation, and projects back to 32 dimensions. Residual connections and layer normalisation are applied around both the attention sub-block and the FFN sub-block.

A dropout rate of 0.3 is applied inside both the attention and FFN sub-blocks. The two encoder layers are applied sequentially, with each one refining the token representations further.

2.4 Mean Pooling and Classification Head

After the final encoder layer, we compute the mean of all 80 token representations along the time axis, producing a single 32-dimensional vector. We chose mean pooling over a learnable [CLS] token because, with training data from only a small number of subjects, the CLS token tended to overfit to subject-specific patterns rather than learning a useful aggregate representation. Mean pooling is parameter-free and gives a stable summary regardless of training set size.

This pooled vector is passed through layer normalisation, dropout, and a fully connected linear layer that outputs 5 values, one per class. A softmax is applied to these logits to obtain a probability distribution over the five activities.

2.5 Training Configuration

The model is trained with backpropagation using cross-entropy loss with label smoothing ($\alpha = 0.1$) between the predicted class probabilities and the ground-truth labels. Label smoothing prevents the model from becoming overconfident on the training set, which is especially useful when the number of distinct subjects is small.

Because the training set is class-imbalanced (e.g. Swaying has roughly twice as many windows as Stepping), we address this at two levels. First, we weight the loss by the inverse class frequency so that under-represented activities receive proportionally higher gradients. Second, we use a weighted random sampler during training: each window is sampled with probability inversely proportional to the frequency of its class, so that every class is seen roughly equally often per epoch regardless of the original distribution. This combination of loss weighting and oversampling ensures that minority classes like Stepping receive sufficient training signal despite having far fewer recording sessions.

We use the AdamW optimiser with decoupled weight decay. Gradients are clipped to a maximum norm of 1.0 per step to prevent occasional large updates from destabilising training. The specific training parameters are listed in Table 2.

The learning rate starts at 5×10^{-4} and is decayed following a cosine schedule over the full 150 epochs. Training stops early if validation accuracy does not improve for 80 consecutive epochs. We save the model checkpoint with the highest validation accuracy and use that checkpoint for all subsequent evaluation.

2.6 Data Augmentation

Because training data comes from a small number of subjects, we apply aggressive augmentation to each training window on every epoch to simulate the kind of variation expected from new subjects. The five transforms are:

1. **Gaussian jitter.** Additive noise sampled from $\mathcal{N}(0, 0.15^2)$ is added independently to every element of the window. This simulates sensor noise variation between recording sessions and devices.

2. **Per-channel amplitude scaling.** Each of the 50 features is multiplied by an independent random factor drawn uniformly from $[0.5, 1.5]$. This forces the model to tolerate differences in sensor calibration and body weight across subjects.

3. **Channel dropout.** Each feature channel is independently zeroed out with probability 0.2. This prevents the model from relying too heavily on any single sensor and improves robustness to occasional sensor failures.

4. **Temporal masking.** A contiguous block of up to $\lfloor W/7 \rfloor$ timesteps (roughly 15% of the window) is randomly selected and set to zero, acting as a regulariser that forces the model to use the full temporal context.

5. **Random time shift.** The window is circularly shifted along the time axis by a random offset of up to $\pm W/8$ timesteps. This makes the model invariant to where exactly within the recording the gait event starts.

Augmentation is applied online during training only; validation and evaluation data are not augmented.

In addition to these per-window transforms, we apply **mixup** at the batch level during training. For each mini-batch, we sample a mixing coefficient $\lambda \sim \mathrm{Beta}(\alpha, \alpha)$ with $\alpha = 0.2$ and blend each input window with a randomly paired window from the same batch: $\tilde{x} = \lambda\, x_i + (1 - \lambda)\, x_j$. The loss is computed as $\lambda\, \mathcal{L}(f(\tilde{x}),\, y_i) + (1 - \lambda)\, \mathcal{L}(f(\tilde{x}),\, y_j)$. With $\alpha = 0.2$ the mixing is usually mild (most λ values are close to 0 or 1), but it still smooths decision boundaries and reduces the tendency to memorise individual training windows.

2.7 Model Summary

Table 3 summarises the architectural choices of the TGT.

3 Evaluation and Results

To be completed after training runs.

4 Design Challenges and Reasoning

4.1 Data Leakage from Window-Level Splitting

The original training pipeline split data at the window level: individual sliding windows were randomly shuffled into train, validation, and test sets. Because windows overlap by a large margin (stride smaller than window size), near-identical windows ended up in both training and validation sets. This made validation accuracy look artificially high (above 87%) while performance on genuinely unseen Group 10 data was around 37%.

We fixed this by implementing file-level splitting (`split_by_file`). All windows from a single CSV recording stay in the same partition, so the model can never see overlapping segments during validation. This is also how we identified that the baseline MLP notebooks had the same leakage problem: their high reported accuracy was not trustworthy on truly unseen subjects.

4.2 Limited Subject Diversity

The training dataset contains recordings from only a handful of distinct subjects. The model quickly memorised subject-specific quirks (resting pressure levels, body weight effects, sensor placement) rather than learning generalisable gait patterns. This is why it performed well internally but collapsed on new subjects.

We addressed this through several layers:

- Incorporating Group 10 data into the training pool to add more subject diversity.
- Per-window z-score normalisation removes subject-specific absolute baselines so the model only sees the temporal shape of each signal, not its magnitude.
- Aggressive augmentation (five transforms) simulates the kind of variation a new subject would introduce: different noise levels, sensor sensitivity, missing channels, shifted timing.
- Small model capacity (32-dimensional embeddings, roughly 40k parameters versus the MLP's 800k) to limit the model's ability to memorise.

4.3 Class Imbalance

Stepping had far fewer recording sessions than classes like Swaying or Injury_walking. With standard training, the model learned to mostly ignore Stepping because it could still achieve high overall accuracy by getting the majority classes right.

We addressed this at two levels. First, inverse-frequency loss weighting gives Stepping errors a proportionally larger gradient signal. Second, a weighted random sampler oversamples minority classes so that every class appears roughly equally often per epoch. This is stronger than loss weighting alone because it changes how often the model actually sees minority examples.

Even with both fixes, Stepping remains the weakest class (F1 around 0.28 to 0.35) because it may genuinely be the most ambiguous activity from insole sensor data alone.

4.4 Architecture: CNN-Transformer Hybrid

The original model was a pure Transformer with a linear projection per timestep and a learnable CLS token. This had two problems. A linear projection treats each timestep independently and misses local temporal patterns such as heel-strike to toe-off pressure waves and acceleration spikes at foot contact. The CLS token, being a single learnable vector, tended to overfit with so few training subjects.

We replaced the linear projection with a 1D CNN stem (two convolutional layers, kernel sizes 7 and 5) that extracts local temporal features before the Transformer sees them. These local features (pressure wave shapes, acceleration peaks) are more consistent across subjects than raw sensor values. We replaced the CLS token with mean pooling, which is parameter-free and more stable with limited data.

4.5 Why a Small Model

The baseline MLP notebooks used roughly 800k parameters. With limited training subjects, a large model has enough capacity to memorise individual subjects rather than learn generalisable patterns. The MLP's high evaluation accuracy was an artefact of data leakage, not genuine generalisation. We deliberately kept the TGT small ($d_{\text{model}} = 32$, 2 heads, 2 layers, 128 FFN hidden dimension) to force the model to learn compact, transferable representations.

4.6 Dual Normalisation

We apply normalisation twice. A global z-score (fit on the training set only) brings all features to comparable scales. A per-window z-score removes within-window absolute offsets so the model learns from relative temporal patterns, not absolute sensor magnitudes that vary per subject.

This two-stage approach was motivated by the observation that subjects differ most in their baseline sensor readings (resting pressure, standing orientation), while the temporal dynamics of each activity are more consistent across subjects.

4.7 Mixup Augmentation

After reaching roughly 91.3% validation accuracy, we added mixup ($\alpha = 0.2$) to push further. Mixup blends pairs of training windows and their labels, smoothing decision boundaries. This caused training accuracy to drop to around 55–65% (expected, since the model is evaluated against hard labels but trained on blended targets), while validation accuracy improved to 92.01%. The gap between training and validation accuracy with mixup is normal and not a sign of a problem.

4.8 Training Stability

Gradient clipping (max norm 1.0) prevents occasional large gradients from destabilising training, which was causing bumpy training curves. Label smoothing (0.1) prevents overconfident predictions, acting as a soft regulariser. A cosine annealing learning rate schedule with high patience (80 epochs) gives the model enough time to converge under heavy augmentation and mixup, which slow convergence. Early stopping on validation accuracy ensures we save the best generalising checkpoint rather than the most overfit one.

4.9 Comparison with Baseline MLP

The baseline MLP notebooks reported very high accuracy on their evaluation sets, but this was misleading due to window-level splitting. Their evaluation windows came from the same recording sessions as training windows, often overlapping by more than 50%. The TGT with file-level splitting produces honest metrics on genuinely held-out recordings, which is why its numbers look lower but are more trustworthy.

Appendix

Insole Feature Definitions

Retained Input Features

Class ID	Label	Description
0	Normal_walking	Baseline gait pattern without perturbations or injury. Used as reference for distinguishing altered gait behaviours.
1	Injury_walking	Walking while simulating or experiencing injury (e.g., limping, reduced load on one side), leading to asymmetric pressure and motion patterns.
2	Stepping	Repeated stepping in place or step-like movements with frequent foot lifts and contacts but limited forward progression.
3	Swaying	Predominantly static stance with body sway, where the centre of pressure moves but feet largely remain planted.
4	Jumping	Dynamic movements with clear take-off and landing phases, characterised by brief loss of contact and high impact forces on landing.

Table 1: Activity classes used for supervised learning. Each 0.33 s window is assigned one of five class IDs according to the folder name of the source recording, following the mapping implemented in the code.

Parameter	Value
Optimiser	AdamW
Learning rate	5×10^{-4}
Weight decay	1×10^{-3}
LR schedule	Cosine annealing
Batch size	64
Max epochs	150
Early stopping patience	80 epochs
Dropout	0.3
Label smoothing	0.1
Gradient clipping	Max norm 1.0
Loss weighting	Inverse class frequency
Training sampler	Weighted random (oversample minority
Mixup α	0.2

Table 2: Training hyperparameters used for TGT.

Component	Value
Input features per timestep	50
Window size (W)	80
Stride (S)	30
Conv stem	2 layers (kernel 7, kernel 5)
Channel dimension (d_{model})	32
Attention heads (H)	2
Encoder layers (N)	2
FFN hidden dimension	128
Positional encoding	Fixed sinusoidal
Sequence aggregation	Mean pooling
Per-window normalisation	z-score per feature
Global normalisation	z-score (fit on train)
Data augmentation	Jitter, scaling, channel dropout, time mask, time shift
Output classes	5

Table 3: TGT model architecture summary.

Signal	Range	Components	Elements	Notes
Cap sensing (pressure)	$0 \ldots 800$	18 electrodes	Ele_0–Ele_17	Per-electrode plantar pressure.
Accelerometer (x, y, z)	$-1 \ldots 1$	x, y, z	Ele_18–Ele_20	3D linear acceleration.
Accelerometer (time)	> 0	timestamp	Ele_21	**Redundant:** inner accelerometer timestamp (same timestep as row, dropped in preprocessing).
Gyroscope (roll, pitch, yaw)	$-600 \ldots 600$	r, p, y	Ele_22–Ele_24	Angular velocity in °/s.
Gyroscope (time)	> 0	timestamp	Ele_25	**Redundant:** inner gyroscope timestamp (same timestep as row, dropped in preprocessing).
Magnetometer (x, y, z)	$-3000 \ldots 3000$	x, y, z	Ele_26–Ele_28	Local magnetic field.
Magnetometer (time)	> 0	timestamp	Ele_29	**Redundant:** inner magnetometer timestamp (same timestep as row, dropped in preprocessing).
Quaternion	$-1 \ldots 1$	x, y, z, v	Ele_30–Ele_33	Orientation computed from all previous sensors.
Quaternion (time)	> 0	timestamp	Ele_34	**Redundant:** inner quaternion timestamp (same timestep as row, dropped in preprocessing).
Unused	—	—	Ele_35	Not used (reserved).
Insole side label	$\{0, 1\}$	right / left	Ele_36	Right = 0, Left = 1.
Battery status	$0 \ldots 100$	percentage	Ele_37	**Redundant:** remaining battery level (not used in the model).

Table 4: Overview of per-timestep insole features, their numeric ranges, and element indices in the raw data vector.

Signal	Range	Components	Elements	Rationale for inclusion
Cap sensing (pressure)	$0 \ldots 800$	18 electrodes	Ele_0–Ele_17	High-resolution plantar pressure map. Captures stance phase, load distribution, and contact pattern differences across activities (e.g., forefoot vs. heel loading in jumping vs. walking).
Accelerometer (linear)	$-1 \ldots 1$	x, y, z	Ele_18–Ele_20	Foot-frame linear acceleration. Encodes impact magnitude, swing dynamics, and vertical motion, which are highly discriminative for gait phase transitions and dynamic activities such as jumping.
Quaternion (orientation)	$-1 \ldots 1$	x, y, z, v	Ele_30–Ele_33	3D orientation of the insole, fused from inertial and magnetic sensors. Provides a numerically stable, gimbal-lock-free representation of foot pose over time, enabling the model to learn differences in foot rotation patterns between gait types.

Table 5: Subset of insole signals retained for model input. We keep per-timestep plantar pressure, linear acceleration, and fused orientation (quaternions) for each foot, as these jointly capture contact mechanics, dynamic motion, and foot pose. Redundant timestamps, raw gyroscope/magnetometer channels, and battery state are excluded from the learning pipeline.